

Team Nucleo — Project Viva Questions and Answers

Use these answers as a speaking guide. Adapt them to the question and be honest about prototype limitations.

Project overview

1. What problem does this project solve?

It reduces the manual first-pass effort of reviewing variants in a VCF file. It ingests the file, organises variants by case, adds public evidence, uses a prototype ML score to prioritise them, and gives the user an evidence view and PDF report. It supports review; it does not replace a geneticist or clinician.

2. Explain the full workflow in one minute.

The user uploads a VCF. Express receives it using Multer, and a stream parser reads each record without loading the whole file into memory. Prisma saves a Patient record and its Variants in SQLite. When analysis is requested, the backend gets allele frequency, CADD, and ClinVar context from MyVariant.info; it batches AF and CADD values to a FastAPI service. The Python Random Forest returns a score, class, and explanation. The backend saves evidence, updates the variant status, and React displays the results and exports a PDF.

3. Why did you build separate frontend, backend, and ML services?

Each layer has a focused responsibility. React is suited to interaction and visualisation; Node/Express handles web uploads and database orchestration; Python has the mature ML ecosystem for scikit-learn. Keeping them separate makes the model replaceable, lets each service be tested independently, and reflects real integration patterns.

4. Why use APIs instead of putting everything in the frontend?

The browser should not contain database credentials, model files, upload storage logic, or external annotation-service handling. APIs create a controlled boundary: the frontend asks for an action, while the server validates it, persists data, calls external systems, and returns only the necessary result.

Biology and genomic data

5. What is a VCF file?

VCF means Variant Call Format. It is a standard text format that stores genomic variants. A record commonly includes chromosome, position, identifier, reference allele, alternate allele, quality, filter, and

information fields. The application needs chromosome, position, reference, and alternate allele to identify a variant.

6. What do REF and ALT mean?

REF is the reference allele—the expected base or sequence at that position in the chosen reference genome. ALT is the observed alternative allele. For example, **G → A** means the reference base is G and the variant base is A.

7. What is pathogenicity?

Pathogenicity describes whether a variant has sufficient evidence to contribute to disease in a specific clinical context. It is not determined by one score alone; context such as phenotype, inheritance, population frequency, quality, and published evidence matters.

8. What is a VUS?

A Variant of Uncertain Significance is a variant with insufficient or conflicting evidence. A VUS is not a positive diagnosis and should not be treated as pathogenic simply because it is uncertain.

9. Why is allele frequency useful?

If an allele is common in the general population, it is less likely to cause a very rare, severe, highly penetrant inherited disease. This is a prioritisation signal, not an absolute rule; frequency must be interpreted with population and disease context.

10. What is CADD?

CADD is a computational framework that ranks variants by predicted deleteriousness. The application uses its Phred-scaled score when available. A higher score can support prioritisation, but it is not clinical proof.

11. What are ClinVar and gnomAD?

ClinVar is a public archive of submitted clinical interpretations of variants. gnomAD is a large population reference database used for allele frequency. MyVariant.info aggregates information from sources such as these and makes it available through an API.

Implementation decisions

12. Why use stream-based parsing for VCFs?

Genomic files can be large. Reading a VCF line by line using Node's **fs** and **readline** keeps memory use controlled. The backend then inserts variants in batches of 1,000 to avoid a huge single database operation.

13. Why use SQLite and Prisma?

SQLite makes a portable local prototype easy to run with no separate database server. Prisma provides a schema, relationships, migrations/schema push tooling, and safer readable ORM queries. For production, PostgreSQL or another managed database would be more appropriate.

14. Why use a Random Forest?

Random Forest works well on tabular numerical data, captures non-linear relationships, gives probability outputs, and is straightforward to train and deploy. It is a reasonable baseline for a prototype. It is not presented as a validated clinical model.

15. What features feed the model?

`allele_frequency` and `cadd_score`. The model outputs a pathogenicity probability. The service maps scores below 0.20 to Benign, 0.20 to below 0.80 to VUS, and 0.80 or above to Pathogenic.

16. Is this model trained on real patient data?

No. The repository's training script generates 10,000 synthetic examples using a biological-inspired formula. This allows the end-to-end architecture to be demonstrated, but it means the printed model accuracy/AUC cannot be claimed as real-world clinical performance.

17. Why use a FastAPI ML service?

FastAPI gives a lightweight typed HTTP interface around the Python model. The Node backend can submit a batch of features and receive JSON predictions without needing Python embedded in the Node process. It also makes future model replacement easier.

18. What does the explanation mean? Is it actually SHAP?

The UI calls it a SHAP explanation, but the current Python code generates a human-readable, rule-based summary from AF, CADD, and the predicted class. It is feature-oriented explanation text, not a full computation of SHAP values. A future implementation should calculate and store genuine SHAP values.

19. Why batch calls to the ML service?

Batching reduces HTTP overhead and lets the ML service evaluate many feature rows in one operation. It is faster and simpler than one HTTP request per variant.

20. Why limit analysis to 50 variants?

It is a prototype responsiveness safeguard. Public annotation requests and sequential database updates could be slow for very large VCFs. A production system would use a background queue, progress tracking, caching, and process all variants safely.

21. What happens if MyVariant.info is unavailable?

The lookup has a short timeout and returns fallback zero values/null ClinVar context on an error. This prevents the interface from hanging. However, a fallback result is weaker evidence and should not be clinically interpreted; production should expose annotation-source status clearly and retry/cache responsibly.

22. How do you protect against an invalid ML response?

The backend allows only `Benign`, `VUS`, and `Pathogenic`. If the ML service returns a missing or unexpected label, it logs a warning and stores `VUS` as the safe fallback rather than saving arbitrary data.

Frontend and reporting

23. What can a user do in the interface?

They can upload a VCF, select a prior case, run analysis, search/filter/sort variants, open a detailed evidence drawer, switch themes, clear local history, and export a structured PDF focused on pathogenic and VUS variants.

24. Why export only Pathogenic and VUS variants to the PDF?

The report is designed as a prioritisation handoff. Benign variants are less likely to require immediate follow-up and are omitted to make the report readable. The summary still shows their total count.

25. How does search work?

The variant table supports a submitted search query across chromosome, position, reference/alternate alleles, classification, and any available gene text, plus explicit status filters. Rows are sorted with pathogenic variants first and then by model score.

26. Why did you focus on UI design?

Genomic review is information-dense. Clear status colour, table hierarchy, readable evidence, and an organised report lower cognitive load. The visual changes do not alter analysis logic; they make the working prototype easier to demonstrate and review.

Security, validation, and limitations

27. Is the project ready for real hospital data?

No. It has no authentication, role-based access, audit trails, encryption strategy, consent workflow, production deployment hardening, or clinical validation. It must be treated as a prototype/demo.

28. How do you validate uploaded files?

Multer limits uploads to 300 MB and the backend accepts `.vcf` files. The parser ignores headers and malformed lines with insufficient columns, then validates chromosome, position, REF, and ALT before storing records.

29. Does the app support compressed VCF files?

Not fully. The frontend currently suggests `.vcf.gz` support, but the backend filter accepts only `.vcf`, and the parser does not decompress gzip streams. This is a known issue we would either implement with a gzip stream or remove from the UI claim.

30. What are the most important current limitations?

Synthetic training data, only two ML features, a 50-variant analysis limit, first-ALT-only handling for multi-allelic records, public annotation dependency, no real SHAP values, and no clinical/security compliance controls.

Future enhancements

31. What would you improve first?

I would make the pipeline robust for realistic data: true gzip support, background job processing with progress, caching/retry of annotations, all-variant processing, and tests around failures. In parallel, I would replace the synthetic training set with curated labelled data under appropriate governance.

32. How could the model be improved?

Add validated features such as consequence/transcript annotations, gene-disease relationships, inheritance/segregation evidence, quality/depth, conservation across species, phenotype similarity, ClinVar review status, and literature evidence. Then use a held-out external test set, probability calibration, subgroup analysis, and clinician review.

33. How would you make it scalable?

Move from SQLite to PostgreSQL, use a task queue such as BullMQ/Celery, store uploads in object storage, add caching for annotations, rate-limit external requests, use job status/progress endpoints, and containerise/deploy the services independently.

34. How would you make it secure?

Add login and role-based access control, HTTPS, encrypted data at rest, secrets management, audit logs, data retention controls, input antivirus/content validation, least-privilege service accounts, and compliance review appropriate to the deployment jurisdiction.

35. How would you make reports more clinically useful?

Add patient/phenotype and sample metadata, reference genome build, transcript and HGVS notation, evidence source/version/date, ACMG/AMP criteria where clinically validated, reviewer sign-off, and a clear distinction between automated prioritisation and final clinical interpretation.

Rapid-fire questions

Question	Short answer
What port is the frontend on?	Usually Vite on 5173.
What port is the backend on?	3001.
What port is the ML service on?	8000.
What is the database?	SQLite accessed through Prisma.
What endpoint starts analysis?	<code>POST /api/analyze/:patientId</code> .
What external annotation service is used?	MyVariant.info.
What is the active ML algorithm?	scikit-learn RandomForestClassifier.

Question	Short answer
What are the ML features?	Allele frequency and CADD score.
What does VUS mean?	Variant of uncertain significance.
What is the safe fallback for an invalid ML label?	VUS.

Good closing statement for a viva

“Our contribution is an end-to-end, explainable prototype workflow—from standard VCF input to evidence-assisted prioritisation and reporting. We deliberately separate the UI, data/orchestration API, and Python ML service so the platform can be tested and extended. We are also explicit that this is not a clinical diagnostic system: the current model uses synthetic data and requires real-data validation, security controls, and expert review before any clinical use.”